

Research Question

Does neural function approximation (Deep CFR) increase or decrease individual best-response exploitability relative to tabular CFR in 3-player Leduc Hold'em, and how does this exploitability evolve over training iterations?

All exploitability measurements assume **no collusion**: the best-responding player deviates unilaterally while the other two players continue playing their fixed approximate equilibrium strategies. This is the standard individual exploitability definition extended to the multiplayer setting.

Definitions & Formal Setup

Individual Exploitability (3-Player, No Collusion)

Let σ be a strategy profile. The **individual exploitability** of player i is:

$$E_i(\sigma)$$

where σ_{-i} denotes the fixed strategies of the other two players and $u_i(\sigma)$ is player i 's expected utility. The **total exploitability** of the profile is:

$$E(\sigma) = \sum_i E_i(\sigma)$$

This differs from the 2-player case only in that we hold **two** opponents fixed rather than one. Since the other players are not coordinating their deviation, this is strictly a no-collusion measurement.

Why This Is the Right Metric

- It is **computable**: best-response against a fixed 2-opponent profile is a single-agent planning problem, solvable exactly in Leduc's small game tree.
- It is **interpretable**: it answers "how much could a single shark gain by perfectly exploiting this approximate equilibrium?"
- It is **comparable across methods**: both tabular CFR and Deep CFR produce strategy profiles, and this metric applies identically to both.
- It **avoids the collusion question entirely**, which is both legally motivated (collusion is against the rules in real poker) and theoretically cleaner.

Hypotheses

H1 (Convergence): Tabular CFR converges to near-zero individual exploitability in 3-player Leduc. This is the known baseline (exploitability $E_i(\sigma)$ after sufficient iterations).

H2 (Neural degradation): Deep CFR exhibits higher asymptotic individual exploitability than tabular CFR due to function approximation error " i.e., the neural network introduces a noise floor below which exploitability cannot fall.

H3 (Oscillation): Deep CFR's per-player exploitability oscillates across training iterations rather than monotonically decreasing, reflecting the absence of convergence guarantees in the multiplayer setting.

H4 (Per-player asymmetry): Exploitability may be unevenly distributed across the three players due to position effects and betting-order asymmetries in Leduc's structure.

Experimental Design

Game Environment

- Game**: 3-player Leduc Hold'em
- Library**: DeepMind open_spiel (handles game tree, CFR baselines, best-response computation out of the box)
- Deck**: 6 cards (J, Q, K ? 2); 2 betting rounds; fixed bet sizes (2 chips pre-flop, 4 chips post-flop); 1 raise cap per round

Baselines

Method	Description
	Standard vanilla CFR; open_spiel reference implementation

Tabular CFR	
Tabular CFR+	CFR with regret floor at zero; faster convergence, use as upper baseline
Random policy	Uniform random actions; establishes exploitability ceiling

Treatment

- Deep CFR** (Brown et al., 2019): replace regret tables with neural networks; use open_spiel’s Deep CFR implementation or the authors’s reference code

Neural Architecture

Keep simple to isolate the effect of approximation vs.Â architecture search:

- 3-layer MLP, 64-128 hidden units per layer
- Input: information set encoding (private card + board card + betting history as one-hot)
- Output: per-action regret estimates
- Train with Adam, learning rate sweep over {1e-3, 3e-4}

Measurements

Primary Measurements (collected every [formula] iterations)

- Total individual exploitability** [formula] “ main result
- Per-player exploitability** [formula] for [formula] “ tests H4
- Policy entropy** per information set “ tracks whether strategies are converging or cycling

Secondary Measurements

- Best-response value by position** (early position, middle position, late position) “ exploitability broken down by acting order
- Regret magnitude** over training “ tabular only; shows whether regret is actually minimized
- Cross-play exploitability:** compute best-response of a tabular CFR player against the Deep CFR profile and vice versa

Compute Budget

Leduc Hold’s em has ~[formula] information sets. All measurements can be run on CPU in minutes. No GPU required for tabular experiments; Deep CFR training is fast at this scale.

Timeline

Weeks 1-2: Foundations & Baseline

Reading (parallel with implementation):

- Zinkevich et al.Â (2007) “ CFR foundations; focus on counterfactual regret and the convergence proof structure. Note exactly where the proof breaks for [formula] players.
- Brown & Sandholm (2019), Pluribus “ skim for multiplayer equilibrium discussion; read Section 2 carefully. Do not implement anything from this paper.

Implementation:

- Install open_spiel; load 3-player Leduc
- Run tabular CFR to convergence; verify exploitability reaches near-zero (this is your sanity check that the stack works)
- Instrument exploitability logging every 100 iterations

Deliverable: Exploitability-vs-iterations plot for tabular CFR. This should match published numbers approximately.

Weeks 3-4: Deep CFR Baseline

Reading:

- Brown et al. (2019), *Deep Counterfactual Regret Minimization* – read fully; focus on Section 3 (the approximation) and Section 5 (empirical results on Leduc)
- Heinrich & Silver (2016), NFSP – read conceptually only; do not implement

Implementation:

- Run `open_spiel`'s Deep CFR on **2-player** Leduc first; verify exploitability matches the Deep CFR paper's reported numbers (~small mbb/g). This confirms correct setup.
- Extend to **3-player** Leduc. This is the novel step – the paper does not report 3-player Leduc results.

Deliverable: Side-by-side exploitability curves: tabular CFR vs. Deep CFR, 3-player Leduc. This is the **core empirical result of the paper**.

Weeks 5-6: Per-Player Analysis & Oscillation Study

Research tasks:

- Decompose total exploitability into per-player components; plot [formula] for each [formula] over training
- Track policy entropy at each information set; identify whether specific infosets oscillate
- Run multiple random seeds (minimum 5) to distinguish noise from systematic instability
- Test H4: does early-position or late-position player suffer more exploitability?

Implementation:

- Write a best-response solver wrapper (or use `open_spiel`'s built-in) that fixes two players' strategies and optimizes the third
- Run this for all three player positions at each checkpoint

Deliverable: Per-player exploitability plots; entropy heatmaps over information sets.

Weeks 7-8: Ablations & Cross-Play

Ablations (pick 2-3 based on what weeks 3-6 reveal):

- Network size: 32 vs. 64 vs. 128 hidden units – does larger network reduce approximation noise floor?
- Training samples: does more self-play data reduce exploitability or just slow convergence?
- CFR variant: vanilla CFR vs. CFR+ as the tabular baseline

Cross-play:

- Compute exploitability of a tabular CFR best-responder against the Deep CFR profile
- This tests whether Deep CFR is exploitable by a *rational* opponent using classical methods

Deliverable: Ablation table; cross-play exploitability comparison.

Weeks 9-10: Writing

Paper structure (workshop format, 4-6 pages):

1. Introduction – the multiplayer exploitability measurement problem; why individual best-response under no-collusion is the right metric
2. Background – CFR, Deep CFR, individual exploitability definition
3. Experimental setup – 3-player Leduc, measurement protocol
4. Results – exploitability curves, per-player breakdown, oscillation evidence
5. Discussion – what the results imply for practical multiplayer solver design
6. Conclusion

Target venues:

- NeurIPS Workshop on Decision-Making and Uncertainty
 - AAAI Workshop on Game Theory and Computation
 - ICML Workshop on Reinforcement Learning in Games
-

What Is NOT In Scope

Excluded	Reason
Coalition exploitability	Legally and practically distinct; separate paper
Full No-Limit Hold'em	Intractable at this scale

ReBeL / belief-state search	Adds complexity without serving the research question
LLM-based agents	Different research direction; crowded space
Implementing CFR from scratch	Use <code>open_spiel</code> ; your contribution is the measurement, not the solver
New equilibrium concepts	Requires deeper game theory background

Engineering Stack

Component	Tool
Game environment	<code>open_spiel</code> (DeepMind, Apache 2.0)
CFR baselines	<code>open_spiel</code> built-in implementations
Deep CFR	<code>open_spiel</code> reference or authors'™ code
Neural networks	PyTorch (MLP only; keep simple)
Exploitability computation	<code>open_spiel</code> best-response utilities
Experiment tracking	Weights & Biases or simple CSV logging
Analysis & plots	Python, matplotlib, pandas

Expected Contributions

1. **Empirical:** First systematic measurement of individual best-response exploitability of Deep CFR in 3-player (vs. 2-player) Leduc Hold'em across training iterations
2. **Methodological:** A clean, reproducible protocol for measuring individual exploitability under the no-collusion assumption in [formula]-player imperfect-information games
3. **Practical:** Evidence for or against neural approximation as a viable path to multiplayer poker solving " directly relevant to the GTO Wizard multiway engine and similar systems

Key Papers (Ordered by Priority)

Priority	Paper	Why
? Essential	Zinkevich et al. (2007)	CFR foundations
? Essential	Brown et al. (2019) Deep CFR	Core method being studied
? Essential	Brown & Sandholm (2019) Pluribus	Multiplayer context
? Important	Johanson et al. (2012) MCCFR	Sampling methods background
? Important	Brown & Sandholm (2017) Libratus	Heads-up baseline context
? Optional	Heinrich & Silver (2016) NFSP	Alternative neural method
? Optional	Brown et al. (2020) ReBeL	Future directions only